

UNIVERSIDAD NACIONAL DE SAN AGUSTÍN

FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS

ESCUELA PROFESIONAL DE CIENCIA DE LA COMPUTACIÓN



BIG DATA

Ejercicios de Hive

Estudiante: Merisabel Ruelas Quenaya

Profesor: Alvaro Mamani Aliaga

AREQUIPA – PERÚ

2025

EJERCICIO 1: WORDCOUNT

SQL

```
CREATE TABLE docs (line STRING);

LOAD DATA LOCAL INPATH '/home/hivehadoop/wordcount.txt'
OVERWRITE INTO TABLE docs;

CREATE TABLE word_count AS

SELECT word, count(1) AS count

FROM (

    SELECT explode(split(line, ' ')) AS word

    FROM docs

) w

GROUP BY word

ORDER BY word;
```

1. CREATE TABLE docs (line STRING);`

Hive crea una tabla administrada llamada docs.

- En el sistema de archivos (por defecto HDFS), se crea una carpeta: /user/hive/warehouse/docs
- Esta tabla tiene una sola columna de tipo STRING, que almacenará cada línea completa del archivo como un solo registro.
- Se registra en el Metastore (la base de datos interna de Hive que almacena metadatos de tablas).
- No se cargan datos aún, solo se crea la estructura.

2. LOAD DATA LOCAL INPATH ... INTO TABLE docs

SQL

```
LOAD DATA LOCAL INPATH '/home/hivehadoop/wordcount.txt'
OVERWRITE INTO TABLE docs;
```

Carga el contenido del archivo wordcount.txt desde el sistema de archivos local (no desde HDFS).

- Cada línea del archivo se convierte en una fila en la tabla docs.
- La opción OVERWRITE borra cualquier dato anterior en la tabla.
- Hive copia el archivo desde /home/hivehadoop/wordcount.txt hacia: /user/hive/warehouse/docs/ en HDFS.
- No interpreta el contenido aún, solo lo almacena como está (líneas de texto).
- Usa el InputFormat predeterminado TextInputFormat, que considera cada línea como un registro.

3. CREATE TABLE word_count AS SELECT ...

SQL

```
CREATE TABLE word_count AS
SELECT word, count(1) AS count
FROM (
  SELECT explode(split(line, ' ')) AS word
  FROM docs
) w
GROUP BY word
ORDER BY word;
```

¿Qué hace?

Este bloque realiza el procesamiento principal del WordCount. Vamos paso por paso:

a) split(line, ' ')

Divide el contenido de cada línea usando espacios como separador.

Resultado: un array de palabras por línea.

Ejemplo:

SQL

```
split('Hola mundo hola', ' ') → ['Hola', 'mundo', 'hola']
```

b) explode(...)

Toma cada array y genera una fila por cada elemento del array.

Ejemplo:

SQL

```
explode(['Hola', 'mundo', 'hola']) →
```

'Hola'

'mundo'

'hola'

Así, de una línea se generan múltiples registros.

c) FROM docs

Hive recorre cada fila de la tabla docs y aplica split() y explode() a cada una.

Resultado: una tabla temporal con muchas filas, una por cada palabra.

d) GROUP BY word

Agrupar todas las filas por la palabra (word), es decir, poner juntas todas las ocurrencias de cada palabra.

e) count(1) AS count

Cuenta cuántas veces aparece cada palabra.

count(1) es una forma eficiente de contar filas por grupo (no depende de contenido nulo).

f) CREATE TABLE word_count AS (...)

Hive almacena el resultado de la consulta como una nueva tabla física llamada word_count.

- Crea la carpeta correspondiente:
/user/hive/warehouse/word_count
- Inserta los resultados como archivos tipo texto plano.

```
hivehadoop@ubuntumery-NBLK-WAX9X: ~  
hivehadoop@ubuntumery-NBLK-WAX9X:~$ beeline -u jdbc:hive2:// -f /home/hivehadoop/wordcounts.hql  
SLF4J: Class path contains multiple SLF4J bindings.  
SLF4J: Found binding in [jar:file:/home/hivehadoop/hive/lib/log4j-slf4j-impl-2.17.1.jar!/org/slf4j/impl/StaticLoggerBinder.class]  
SLF4J: Found binding in [jar:file:/home/hivehadoop/hadoop/share/hadoop/common/lib/slf4j-reload4j-1.7.35.jar!/org/slf4j/impl/StaticLoggerBinder.class]  
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.  
SLF4J: Actual binding is of type [org.apache.logging.slf4j.Log4jLoggerFactory]  
Connecting to jdbc:hive2://  
Hive Session ID = f3017d8b-7883-4556-8c31-2c5ca38ef6b5  
25/06/04 18:19:19 [main]: WARN session.SessionState: METASTORE_FILTER_HOOK will be ignored, since hive.security.authorization.manager is set to instance of HiveAuthorizerFactory.  
25/06/04 18:19:20 [main]: WARN metastore.ObjectStore: datanucleus.autoStartMechanismMode is set to unsupported value null . Setting it to value: ignored  
25/06/04 18:19:20 [main]: WARN util.DriverDataSource: Registered driver with driverClassName=org.apache.derby.jdbc.EmbeddedDriver was not found, trying direct instantiation.  
25/06/04 18:19:21 [main]: WARN util.DriverDataSource: Registered driver with driverClassName=org.apache.derby.jdbc.EmbeddedDriver was not found, trying direct instantiation.  
25/06/04 18:19:21 [main]: WARN DataNucleus.Metadata: Metadata has jdbc-type of null yet this is not valid . Ignored  
25/06/04 18:19:21 [main]: WARN DataNucleus.Metadata: Metadata has jdbc-type of null yet this is not valid . Ignored  
25/06/04 18:19:21 [main]: WARN DataNucleus.Metadata: Metadata has jdbc-type of null yet this is not valid . Ignored  
25/06/04 18:19:21 [main]: WARN DataNucleus.Metadata: Metadata has jdbc-type of null yet this is not valid . Ignored  
25/06/04 18:19:21 [main]: WARN DataNucleus.Metadata: Metadata has jdbc-type of null yet this is not valid . Ignored  
25/06/04 18:19:21 [main]: WARN DataNucleus.Metadata: Metadata has jdbc-type of null yet this is not valid . Ignored  
25/06/04 18:19:23 [main]: WARN DataNucleus.Metadata: Metadata has jdbc-type of null yet this is not valid . Ignored  
25/06/04 18:19:23 [main]: WARN DataNucleus.Metadata: Metadata has jdbc-type of null yet this is not valid . Ignored  
25/06/04 18:19:23 [main]: WARN DataNucleus.Metadata: Metadata has jdbc-type of null yet this is not valid . Ignored  
25/06/04 18:19:23 [main]: WARN DataNucleus.Metadata: Metadata has jdbc-type of null yet this is not valid . Ignored  
25/06/04 18:19:23 [main]: WARN DataNucleus.Metadata: Metadata has jdbc-type of null yet this is not valid . Ignored  
25/06/04 18:19:23 [main]: WARN DataNucleus.Metadata: Metadata has jdbc-type of null yet this is not valid . Ignored  
Connected to: Apache Hive (version 3.1.3)  
Driver: Hive JDBC (version 3.1.3)  
Transaction isolation: TRANSACTION_REPEATABLE_READ  
0: jdbc:hive2://>  
0: jdbc:hive2://> DROP TABLE IF EXISTS word_count;  
OK  
No rows affected (2.468 seconds)  
0: jdbc:hive2://>  
0: jdbc:hive2://> DROP TABLE IF EXISTS docs;  
OK
```



```

visit_time STRING,
query STRING
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY '\t' ;
LOAD DATA LOCAL INPATH '/home/hivehadoop/excite-small.log'
OVERWRITE INTO TABLE logs;
CREATE TABLE result AS
SELECT user_a, COUNT(1) AS log_entries
FROM logs
GROUP BY user_a
ORDER BY user_a;
SELECT * FROM result;

```

1. DROP TABLE IF EXISTS logs; y DROP TABLE IF EXISTS result;

- Elimina las tablas logs y result si ya existen.
- Esto evita errores en caso de que las ejecuciones anteriores hayan creado estas tablas.

2. CREATE TABLE logs (...)

SQL

```

CREATE TABLE logs (
    user_a STRING,
    visit_time STRING,
    query STRING
)

```

```
ROW FORMAT DELIMITED
```

```
FIELDS TERMINATED BY '\t';
```

- Crea una tabla llamada logs con las siguientes columnas:
 - user_a: ID del usuario
 - visit_time: fecha y hora de la búsqueda
 - query: cadena de búsqueda que ingresó el usuario
- ROW FORMAT DELIMITED: indica que se usará un delimitador de campo.
- FIELDS TERMINATED BY el delimitador es un ****tabulador**** (\t), típico en archivos de logs como este.

Se crea en HDFS: /user/hive/warehouse/logs

3. LOAD DATA LOCAL INPATH ... INTO TABLE logs

SQL

```
LOAD DATA LOCAL INPATH '/home/hivehadoop/excite-small.log'
```

```
OVERWRITE INTO TABLE logs;
```

- Carga los datos del archivo excite-small.log desde el sistema de archivos local a la tabla logs.

Procesos que ocurren:

- Se copia el archivo al HDFS.
- Cada línea se interpreta como un registro.
- Los campos se separan por tabulaciones y se asignan a user_a, visit_time y query.

4. CREATE TABLE result AS SELECT ...

SQL

```
CREATE TABLE result AS
```

```
SELECT user_a, COUNT(1) AS log_entries
```

```
FROM logs
```



```
GROUP BY user_a
```

```
ORDER BY user_a;
```

- Crea una nueva tabla result con el conteo de entradas de logs por cada usuario.

Detalles del proceso:

1. FROM logs: lee todos los registros de la tabla logs.
2. GROUP BY user_a: agrupa las entradas por usuario.
3. COUNT(1): cuenta cuántos registros tiene cada usuario.
4. ORDER BY user_a: ordena alfabéticamente los resultados por ID de usuario.

Se crea la tabla física result con dos columnas:

- user_a: identificador del usuario
- log_entries: número total de registros para ese usuario

5. SELECT * FROM result;

- Muestra todos los resultados almacenados en la tabla result.

Resultado esperado: Una tabla con cada user_a seguido del número de veces que aparece en el archivo de logs (es decir, cuántas búsquedas realizó).


```
hivehadoop@ubuntumery-NBLK-WAX9X: ~
2025-06-04 18:37:18,233 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 2.98 sec
2025-06-04 18:37:25,578 Stage-1 map = 100%, reduce = 100%, Cumulative CPU 5.93 sec
MapReduce Total cumulative CPU time: 5 seconds 930 msec
Ended Job = job_1749078606975_0006
Launching Job 2 out of 2
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
25/06/04 18:37:27 [HiveServer2-Background-Pool: Thread-68]: WARN mapreduce.JobResourceUploader: Hadoop co
mmand-line option parsing not performed. Implement the Tool interface and execute your application with T
oolRunner to remedy this.
Starting Job = job_1749078606975_0007, Tracking URL = http://ubuntumery-NBLK-WAX9X:8088/proxy/application
_1749078606975_0007/
Kill Command = /home/hivehadoop/hadoop/bin/mapred job -kill job_1749078606975_0007
Hadoop job information for Stage-2: number of mappers: 1; number of reducers: 1
25/06/04 18:37:40 [HiveServer2-Background-Pool: Thread-68]: WARN mapreduce.Counters: Group org.apache.had
oop.mapred.Task$Counter is deprecated. Use org.apache.hadoop.mapreduce.TaskCounter instead
2025-06-04 18:37:40,785 Stage-2 map = 0%, reduce = 0%
2025-06-04 18:37:47,084 Stage-2 map = 100%, reduce = 0%, Cumulative CPU 2.49 sec
2025-06-04 18:37:52,302 Stage-2 map = 100%, reduce = 100%, Cumulative CPU 5.46 sec
MapReduce Total cumulative CPU time: 5 seconds 460 msec
Ended Job = job_1749078606975_0007
Moving data to directory hdfs://localhost:9000/user/hive/warehouse/result
25/06/04 18:37:54 [HiveServer2-Background-Pool: Thread-68]: WARN metastore.ObjectStore: datanucleus.autos
tartMechanismMode is set to unsupported value null . Setting it to value: ignored
25/06/04 18:37:54 [HiveServer2-Background-Pool: Thread-68]: WARN metastore.ObjectStore: datanucleus.autos
tartMechanismMode is set to unsupported value null . Setting it to value: ignored
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1 Reduce: 1 Cumulative CPU: 5.93 sec HDFS Read: 12211 HDFS Write: 200 SUCCESS
Stage-Stage-2: Map: 1 Reduce: 1 Cumulative CPU: 5.46 sec HDFS Read: 7346 HDFS Write: 126 SUCCESS
Total MapReduce CPU Time Spent: 11 seconds 390 msec
OK
No rows affected (55.433 seconds)
0: jdbc:hive2://>
0: jdbc:hive2://> SELECT * FROM result;
25/06/04 18:37:54 [b148a922-1d38-463f-a239-ed5569819009 main]: WARN metastore.ObjectStore: datanucleus.au
toStartMechanismMode is set to unsupported value null . Setting it to value: ignored
OK
+-----+-----+
| result.user_a | result.log_entries |
+-----+-----+
| 2A9EABFB35FB954 | 1 |
| 82F4F13FA37520BF | 2 |
| BED75271605EBD0C | 3 |
+-----+-----+
3 rows selected (0.327 seconds)
0: jdbc:hive2://>
0: jdbc:hive2://> Closing: 0: jdbc:hive2://
```

EJERCICIO 3 Identificar cuales usuarios visitan paginas mejores rankeadas en promedio

1. CREATE TABLE pages (...)

Hive crea una tabla administrada llamada pages con tres columnas:

- name de tipo STRING
- url de tipo STRING
- time de tipo STRING
- ROW FORMAT DELIMITED: Indica que es un formato de texto delimitado

- **FIELDS TERMINATED BY ' ':** Define el espacio como separador de campos
- Se crea la carpeta en HDFS: `/user/hive/warehouse/pages`
- Se registran los metadatos en el Metastore de Hive
- No se cargan datos aún, solo se define la estructura

2. **LOAD DATA LOCAL INPATH ... INTO TABLE** `pages`

SQL

```
LOAD DATA LOCAL INPATH '/home/hivehadoop/visits.log'
OVERWRITE INTO TABLE pages;
```

Carga datos desde un archivo local al sistema de archivos de HDFS y los asocia a la tabla.

- **LOCAL INPATH:** Indica que el archivo está en el sistema de archivos local
- **OVERWRITE:** Elimina cualquier dato existente en la tabla antes de cargar los nuevos
- Copia el archivo `visits.log` a HDFS en: `/user/hive/warehouse/pages/`
- Interpreta cada línea del archivo como un registro
- Separa los campos usando espacios (como se definió en `CREATE TABLE`)
- Asigna los valores a las columnas en orden: `name`, `url`, `time`

3. **CREATE TABLE** `avg_visit` **AS SELECT ...**

SQL

```
CREATE TABLE avg_visit AS
SELECT name, count(*) AS num_pages
FROM pages
GROUP BY name;
```

Crea una nueva tabla con estadísticas de visitas por nombre.

1. **FROM pages:** Lee todos los registros de la tabla `pages`
2. **GROUP BY name:** Agrupa los registros por el campo `name`
3. **count(*) AS num_pages:** Cuenta cuántos registros hay en cada grupo
4. **CREATE TABLE avg_visit AS:** Almacena el resultado en una nueva tabla física

La tabla `avg_visit` contendrá:

- Una columna name (los valores únicos de la tabla original)
- Una columna num_pages (el conteo de registros para cada nombre)

```
hivehadoop@ubuntumery-NBLK-WAX9X: ~
hivehadoop@ubuntumery-NBLK-WAX9X:~$ beeline -u jdbc:hive2:// -f /home/hivehadoop/avg_visits.hql
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/home/hivehadoop/hive/lib/log4j-slf4j-impl-2.17.1.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/home/hivehadoop/hadoop/share/hadoop/common/lib/slf4j-reload4j-1.7.35.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.apache.logging.slf4j.Log4jLoggerFactory]
Connecting to jdbc:hive2://
Hive Session ID = fd46aff1-4f79-4f59-b041-6b3acaf2bfc1
25/06/04 18:25:52 [main]: WARN session.SessionState: METASTORE_FILTER_HOOK will be ignored, since hive.security.authorization.manager is set to instance of HiveAuthorizerFactory.
25/06/04 18:25:52 [main]: WARN metastore.ObjectStore: datanucleus.autoStartMechanismMode is set to unsupported value null . Setting it to value: ignored
25/06/04 18:25:53 [main]: WARN util.DriverDataSource: Registered driver with driverClassName=org.apache.derby.jdbc.EmbeddedDriver was not found, trying direct instantiation.
25/06/04 18:25:53 [main]: WARN util.DriverDataSource: Registered driver with driverClassName=org.apache.derby.jdbc.EmbeddedDriver was not found, trying direct instantiation.
25/06/04 18:25:54 [main]: WARN DataNucleus.Metadata: Metadata has jdbc-type of null yet this is not valid
. Ignored
25/06/04 18:25:54 [main]: WARN DataNucleus.Metadata: Metadata has jdbc-type of null yet this is not valid
. Ignored
25/06/04 18:25:54 [main]: WARN DataNucleus.Metadata: Metadata has jdbc-type of null yet this is not valid
. Ignored
25/06/04 18:25:54 [main]: WARN DataNucleus.Metadata: Metadata has jdbc-type of null yet this is not valid
. Ignored
25/06/04 18:25:54 [main]: WARN DataNucleus.Metadata: Metadata has jdbc-type of null yet this is not valid
. Ignored
25/06/04 18:25:54 [main]: WARN DataNucleus.Metadata: Metadata has jdbc-type of null yet this is not valid
. Ignored
25/06/04 18:25:56 [main]: WARN DataNucleus.Metadata: Metadata has jdbc-type of null yet this is not valid
. Ignored
25/06/04 18:25:56 [main]: WARN DataNucleus.Metadata: Metadata has jdbc-type of null yet this is not valid
. Ignored
25/06/04 18:25:56 [main]: WARN DataNucleus.Metadata: Metadata has jdbc-type of null yet this is not valid
. Ignored
25/06/04 18:25:56 [main]: WARN DataNucleus.Metadata: Metadata has jdbc-type of null yet this is not valid
. Ignored
25/06/04 18:25:56 [main]: WARN DataNucleus.Metadata: Metadata has jdbc-type of null yet this is not valid
. Ignored
25/06/04 18:25:56 [main]: WARN DataNucleus.Metadata: Metadata has jdbc-type of null yet this is not valid
. Ignored
Connected to: Apache Hive (version 3.1.3)
Driver: Hive JDBC (version 3.1.3)
Transaction isolation: TRANSACTION_REPEATABLE_READ
0: jdbc:hive2://> DROP TABLE IF EXISTS pages;
OK
No rows affected (2.496 seconds)
0: jdbc:hive2://> DROP TABLE IF EXISTS avg_visit;
OK
No rows affected (0.211 seconds)
0: jdbc:hive2://>
```

```
hivehadoop@ubuntumery-NBLK-WAX9X: ~  
toStartMechanismMode is set to unsupported value null . Setting it to value: ignored  
25/06/04 18:26:05 [HiveServer2-Background-Pool: Thread-68]: WARN ql.Driver: Hive-on-MR is deprecated in H  
ive 2 and may not be available in the future versions. Consider using a different execution engine (i.e.  
spark, tez) or using Hive 1.X releases.  
Query ID = hivehadoop_20250604182603_770eeff8-e707-4d61-851d-6288fd7f51e5  
Total jobs = 1  
Launching Job 1 out of 1  
Number of reduce tasks not specified. Estimated from input data size: 1  
In order to change the average load for a reducer (in bytes):  
  set hive.exec.reducers.bytes.per.reducer=<number>  
In order to limit the maximum number of reducers:  
  set hive.exec.reducers.max=<number>  
In order to set a constant number of reducers:  
  set mapreduce.job.reduces=<number>  
WARN : Hive-on-MR is deprecated in Hive 2 and may not be available in the future versions. Consider usin  
g a different execution engine (i.e. spark, tez) or using Hive 1.X releases.  
25/06/04 18:26:06 [HiveServer2-Background-Pool: Thread-68]: WARN mapreduce.JobResourceUploader: Hadoop co  
mmand-line option parsing not performed. Implement the Tool interface and execute your application with T  
oolRunner to remedy this.  
Starting Job = job_1749078606975_0003, Tracking URL = http://ubuntumery-NBLK-WAX9X:8088/proxy/application  
_1749078606975_0003/  
Kill Command = /home/hivehadoop/hadoop/bin/mapred job -kill job_1749078606975_0003  
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1  
25/06/04 18:26:17 [HiveServer2-Background-Pool: Thread-68]: WARN mapreduce.Counters: Group org.apache.had  
oop.mapred.Task$Counter is deprecated. Use org.apache.hadoop.mapreduce.TaskCounter instead  
2025-06-04 18:26:17,944 Stage-1 map = 0%, reduce = 0%  
2025-06-04 18:26:24,313 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 3.14 sec  
2025-06-04 18:26:31,632 Stage-1 map = 100%, reduce = 100%, Cumulative CPU 6.98 sec  
MapReduce Total cumulative CPU time: 6 seconds 980 msec  
Ended Job = job_1749078606975_0003  
Moving data to directory hdfs://localhost:9000/user/hive/warehouse/avg_visit  
25/06/04 18:26:32 [HiveServer2-Background-Pool: Thread-68]: WARN metastore.ObjectStore: datanucleus.autos  
tartMechanismMode is set to unsupported value null . Setting it to value: ignored  
25/06/04 18:26:32 [HiveServer2-Background-Pool: Thread-68]: WARN metastore.ObjectStore: datanucleus.autos  
tartMechanismMode is set to unsupported value null . Setting it to value: ignored  
MapReduce Jobs Launched:  
Stage-Stage-1: Map: 1 Reduce: 1 Cumulative CPU: 6.98 sec HDFS Read: 12592 HDFS Write: 86 SUCCESS  
Total MapReduce CPU Time Spent: 6 seconds 980 msec  
OK  
No rows affected (29.853 seconds)  
0: jdbc:hive2://>  
0: jdbc:hive2://> SELECT * FROM avg_visit;  
25/06/04 18:26:33 [f11d93c4-8f35-41c4-9e0c-cbf60908c456 main]: WARN metastore.ObjectStore: datanucleus.au  
toStartMechanismMode is set to unsupported value null . Setting it to value: ignored  
OK  
+-----+-----+  
| avg_visit.name | avg_visit.num_pages |  
+-----+-----+  
| Amy            | 4                    |  
| Fred           | 2                    |  
+-----+-----+  
2 rows selected (0.292 seconds)
```

EJERCICIO 4: Identificar cuáles usuarios visitan paginas mejores rankeadas en promedio

SQL

```
SET hive.auto.convert.join = false;  
  
DROP TABLE IF EXISTS visits;  
  
CREATE TABLE visits (
```

```

    name STRING,
    url STRING,
    `time` STRING
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ' ';
LOAD DATA LOCAL INPATH '/home/hivehadoop/visits.log'
OVERWRITE INTO TABLE visits;
DROP TABLE IF EXISTS pages;
CREATE TABLE pages (
    url STRING,
    pagerank DECIMAL(3,2)
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ' ';
LOAD DATA LOCAL INPATH '/home/hivehadoop/pages.log'
OVERWRITE INTO TABLE pages;
DROP TABLE IF EXISTS rank_result;
CREATE TABLE rank_result AS
SELECT pr.name
FROM (
    SELECT v.name, AVG(p.pagerank) AS prank
    FROM visits v

```



```
JOIN pages p ON (v.url = p.url)

GROUP BY v.name

) pr

WHERE pr.prank > 0.5;
```

1. SET hive.auto.convert.join = false;

- Esta línea desactiva la conversión automática de JOINS a MapJoins (una optimización interna).
- Esto asegura que los JOINS se ejecuten como reparticiones completas, útil para evitar errores si las tablas son grandes.

2. DROP TABLE IF EXISTS visits;

SQL

```
CREATE TABLE visits (

  name STRING,

  url STRING,

  `time` STRING

)

ROW FORMAT DELIMITED

FIELDS TERMINATED BY ' ';
```

Elimina la tabla visits si ya existe, para evitar errores por duplicación.

- Crea la tabla visits con los siguientes campos:
 - name: nombre del usuario
 - url: página que visitó
 - time: fecha/hora de la visita (el campo se pone entre comillas por ser palabra reservada)
- ROW FORMAT DELIMITED: indica que los campos están separados por un delimitador.
- FIELDS TERMINATED BY ' ': el delimitador es un espacio. Se crea en HDFS: /user/hive/warehouse/visits

3. LOAD DATA LOCAL INPATH '/home/hivehadoop/visits.log' ...

- Carga los datos del archivo visits.log en la tabla visits.
- LOCAL indica que el archivo está en el sistema local (no en HDFS).
- OVERWRITE reemplaza cualquier dato anterior.

Copia visits.log a la carpeta de la tabla en HDFS y lo parsea usando espacio como delimitador.

4. DROP TABLE IF EXISTS pages;

SQL

```
CREATE TABLE pages (  
  url STRING,  
  pagerank DECIMAL(3,2)  
)  
  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ' ';
```

- Elimina la tabla pages si existe.
- Crea la tabla pages con:
 - url: URL de la página
 - pagerank: ranking de importancia de la página (máximo 9.99)

Se crea /user/hive/warehouse/pages

5. LOAD DATA LOCAL INPATH '/home/hivehadoop/pages.log' ...

- Carga los datos del archivo pages.log en la tabla pages.
- Los campos se separan por espacios.
- Se espera que el archivo tenga líneas como: http://x.com 0.85

6. DROP TABLE IF EXISTS rank_result;

SQL

```
CREATE TABLE rank_result AS
```

```
SELECT pr.name
FROM (
  SELECT v.name, AVG(p.pagerank) AS prank
  FROM visits v
  JOIN pages p ON (v.url = p.url)
  GROUP BY v.name
) pr
WHERE pr.prank > 0.5;
```

- Elimina la tabla rank_result si existe.
 - Crea la tabla rank_result con los nombres de usuarios cuyo promedio de PageRank es mayor a 0.5.
1. JOIN entre visits y pages usando la URL como clave.
 2. Agrupa por nombre de usuario (v.name) y calcula el promedio de PageRank (AVG(p.pagerank)).
 3. Filtra solo los usuarios con promedio mayor a 0.5.
 4. Crea una nueva tabla con los resultados.


```
hivehadoop@ubuntumery-NBLK-WAX9X: ~  
25/06/04 19:31:13 [HiveServer2-Background-Pool: Thread-81]: WARN mapreduce.Counters: Group org.apache.hadoop.mapred.Task$Counter is deprecated. Use org.apache.hadoop.mapreduce.TaskCounter instead  
2025-06-04 19:31:13,452 Stage-1 map = 0%, reduce = 0%  
2025-06-04 19:31:14,508 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 71.69 sec  
2025-06-04 19:31:20,738 Stage-1 map = 100%, reduce = 100%, Cumulative CPU 73.92 sec  
MapReduce Total cumulative CPU time: 1 minutes 13 seconds 920 msec  
Ended Job = job_1749078606975_0008  
Launching Job 2 out of 2  
Number of reduce tasks not specified. Estimated from input data size: 1  
In order to change the average load for a reducer (in bytes):  
  set hive.exec.reducers.bytes.per.reducer=<number>  
In order to limit the maximum number of reducers:  
  set hive.exec.reducers.max=<number>  
In order to set a constant number of reducers:  
  set mapreduce.job.reduces=<number>  
25/06/04 19:31:22 [HiveServer2-Background-Pool: Thread-81]: WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application with ToolRunner to remedy this.  
Starting Job = job_1749078606975_0009, Tracking URL = http://ubuntumery-NBLK-WAX9X:8088/proxy/application_1749078606975_0009/  
Kill Command = /home/hivehadoop/hadoop/bin/mapred job -kill job_1749078606975_0009  
Hadoop job information for Stage-2: number of mappers: 1; number of reducers: 1  
25/06/04 19:31:33 [HiveServer2-Background-Pool: Thread-81]: WARN mapreduce.Counters: Group org.apache.hadoop.mapred.Task$Counter is deprecated. Use org.apache.hadoop.mapreduce.TaskCounter instead  
2025-06-04 19:31:33,116 Stage-2 map = 0%, reduce = 0%  
2025-06-04 19:31:38,308 Stage-2 map = 100%, reduce = 0%, Cumulative CPU 1.83 sec  
2025-06-04 19:31:43,473 Stage-2 map = 100%, reduce = 100%, Cumulative CPU 5.41 sec  
MapReduce Total cumulative CPU time: 5 seconds 410 msec  
Ended Job = job_1749078606975_0009  
Moving data to directory hdfs://localhost:9000/user/hive/warehouse/rank_result  
25/06/04 19:31:45 [HiveServer2-Background-Pool: Thread-81]: WARN metastore.ObjectStore: datanucleus.autoStartMechanismMode is set to unsupported value null . Setting it to value: ignored  
25/06/04 19:31:45 [HiveServer2-Background-Pool: Thread-81]: WARN metastore.ObjectStore: datanucleus.autoStartMechanismMode is set to unsupported value null . Setting it to value: ignored  
MapReduce Jobs Launched:  
Stage-Stage-1: Map: 2 Reduce: 1 Cumulative CPU: 73.92 sec HDFS Read: 16837 HDFS Write: 147 SUCCESS  
Stage-Stage-2: Map: 1 Reduce: 1 Cumulative CPU: 5.41 sec HDFS Read: 9322 HDFS Write: 79 SUCCESS  
Total MapReduce CPU Time Spent: 1 minutes 19 seconds 330 msec  
OK  
No rows affected (111.682 seconds)  
0: jdbc:hive2://>  
0: jdbc:hive2://> SELECT * FROM rank_result;  
25/06/04 19:31:45 [16110713-0979-4e27-9097-c5cd0204c3ac main]: WARN metastore.ObjectStore: datanucleus.autoStartMechanismMode is set to unsupported value null . Setting it to value: ignored  
OK  
+-----+  
| rank_result.name |  
+-----+  
| Amy |  
+-----+  
1 row selected (0.222 seconds)  
0: jdbc:hive2://>
```